

RQ3 설계안: Distribution-Agnostic 총화 샘플링

2026-04-16 확정. 7가지 방법 전수 비교.

연구 질문

RQ3: 데이터 분포를 사전에 모를 때, 공간 인식 총화 샘플링(KM20)의 이점을 얼마나 회수할 수 있는가?

가설

H3: Distribution-agnostic 방법들은 사전 클러스터링(KM20) 대비 유의미한 비율의 개선 효과를 회수하면서도, 사전 학습 불필요 또는 극소 비용이라는 운영 이점을 제공한다.

핵심 지표: Recovery Rate

$$\text{recovery_rate} = (\text{방법X} - \text{RANDOM20}) / (\text{KM20} - \text{RANDOM20})$$

- 1.0 = KM20과 동일 (oracle 수준)
- 0.0 = RANDOM20과 동일 (공간 인식 없음)
- 음수 = RANDOM20보다 나쁨

RQ3의 출발점: KM20의 한계

RQ2에서 KM20이 효과적임을 확인했으나, 실용화에 4가지 한계가 있다:

한계	설명
사전 계산 필요	k-means를 전체 데이터에 돌려야 함 (1M: ~30초, 8M: ~분 단위)
stratum_id 저장	모든 행에 클러스터 라벨 컬럼 추가 필요
데이터 변경 시 재계산	INSERT/UPDATE 시 k-means 재실행 or 근사 할당
K 선택 문제	K=200이 최적인지 모름. 데이터마다 다를 수 있음

이 한계를 극복하는 방법을 체계적으로 탐색하는 것이 RQ3.

초기 후보 D (Exqutor Hybrid) 제외 이유

초기 브레인스토밍에서 "Exqutor Adaptive + 우리 총화 하이브리드"(D)를 검토했으나 제외: 1. Phase 7에서 Adaptive update path SIGSEGV 크래시 → 디버깅 필요 2. 디버깅해도 Phase 7 negative finding (hook_est 기반 STRAT가 BERN보다 나쁨) 3. "Exqutor 버그 수정"은 RQ3의 취지(새로운 방법론 제안)와 안 맞음 4. 구현 난이도 최상, 6주 내 비현실적

7가지 방법 평가 매트릭스

	A. LSH	B. KDE	C. RandProj	E. Hilbert	F. MiniBatch	G. Shell	H. Import
이론적 정당성	강	강	강 (JL)	최강 (30년)	중	중	강
기대 recovery	30~60%	50~80%	10~40%	20~60%	75~95%	25~50%	30~70%
구현 난이도	중	상	하	중	하	중	상
6주 내 가능	✓	⚠	✓	✓	✓	✓	⚠
사전 학습	해시 1회	불필요	사영 1회	index 1회	1~5%	불필요	불필요
INSERT 대응	O(1)	자동	O(1)	O(1)	O(d·K)	자동	자동
논문 기여도	높음	높음	중	최고	중	중	높음
RQ1/2 연결	자연	자연	자연	자연	자연	자연	자연

3가지 패러다임 × 7가지 방법

패러다임 1: Offline Partition (데이터 로드 시 1회 계산)

C. Random Projection

- 이론: Johnson-Lindenstrauss lemma — 랜덤 방향 사영이 거리를 ϵ 이내로 보존
- 메커니즘: 랜덤 단위벡터 r 생성 → 각 벡터 v 의 사영값 $v \cdot r$ 계산 → K등분
- 구현: 내적 1회, 가장 단순
- 기대 recovery: 10~40% (1D로 다차원 구조 포착 한계, RANDOM20과 차이 없을 수도)
- 역할: 최단순 하한선

A. LSH (Locality-Sensitive Hashing)

- 이론: 해시 충돌 확률이 거리에 반비례 → 가까운 벡터가 같은 버킷
- 메커니즘: 랜덤 하이퍼플레인 p 개 → 2^p 버킷 → 비례 배분
- 구현: 해시 함수 ~30줄
- 기대 recovery: 30~60%
- 역할: 확률적 공간 포착

E. Hilbert Curve (Space-Filling Curve)

- 이론: Hilbert 곡선은 d 차원 → 1D 매핑에서 공간 인접성을 최대한 보존. R-tree, 공간 DB에서 30년간 검증.
- 메커니즘: 벡터 양자화 → Hilbert index 계산 → index 기준 K등분
- 구현: hilbertcurve 라이브러리 + 양자화
- 기대 recovery: 20~60% (고차원에서 curse of dimensionality로 하향 가능)
- 역할: 결정론적 공간 포착. 벡터 카디널리티 추정에 최초 적용.
- 주의: 96d/128d에서 locality 보존이 약해질 수 있음 → negative finding도 기여

F. Mini-batch K-means

- 이론: Sculley 2010 — 데이터의 1~5%만 보고 근사 클러스터링
- 메커니즘: sklearn MiniBatchKMeans → 센트로이드 학습 → 나머지 $O(d \cdot K)$ 할당
- 구현: sklearn 한 줄
- 기대 recovery: 75~95% (근사 k-means이므로 KM20에 근접)
- 역할: "얼마만큼의 사전 학습으로 충분한가"에 대한 답. 비용-효과 상한선.

패러다임 2: Online Query-Adaptive (쿼리 시점 동적 적응)

G. Distance-Shell

- **이론:** 쿼리 벡터 중심 동심원 분할. 카디널리티 조건 충족 행이 내부 shell에 집중.
- **메커니즘:** pilot 샘플(~385개) → 거리 분위수로 K개 shell → shell별 비례 배분 본 샘플
- **구현:** 2-pass (pilot → 본 샘플)
- **기대 recovery:** 25~50%
- **역할:** 단순 적응형. B(KDE)의 단순화 버전.

B. KDE-pilot (Kernel Density Estimation)

- **이론:** Silverman bandwidth로 비모수 밀도 추정 → Neyman 최적 배분
- **메커니즘:** pilot 샘플 → KDE로 거리 분포 밀도 추정 → 밀도 높은 영역에 더 많은 샘플 배분
- **구현:** scipy KDE + 2-pass
- **기대 recovery:** 50~80% (쿼리별 최적화이므로 이론적으로 가장 정교)
- **역할:** 정교한 적응형. 이론적 상한.

패러다임 3: Weight-based (파티션 없이 가중치 조정)

H. Importance Sampling

- **이론:** 최적 importance weight는 $f(x)/g(x)$ 에 비례. 층 분할 없이 각 샘플의 가중치를 조정하여 분산 감소.
 - **메커니즘:** BERNOULLI로 샘플 추출 → 각 샘플의 로컬 밀도 추정 → 밀도 역수 가중치로 보정된 카디널리티 추정
 - **구현:** hook_est 반환 공식 수정 (가중 평균)
 - **기대 recovery:** 30~70% (가중치 추정 정확도에 의존)
 - **역할:** 비분할 패러다임 대표. 층화와 근본적으로 다른 접근.
-

9-way 비교 체계

보정 없음	BERNOULLI (Exqutor 원본)
무의미 하한	RANDOM20 (무작위 분할)
Offline Partition	├ C. Random Projection (최단순)
	├ A. LSH (확률적)
	├ E. Hilbert Curve (결정론적)
	├ F. Mini-batch KM (근사 학습)
Online Adaptive	├ G. Distance-Shell (단순 적응)
	├ B. KDE-pilot (정교한 적응)
Weight-based	├ H. Importance Sampling (비분할)
오라클 상한	KM20 (사전 클러스터링)

예상 순서: $\text{RANDOM20} \leq C \leq \{A, E, G\} \leq \{B, H\} \leq F \leq \text{KM20}$

실험 설계

데이터셋

- DEEP 1M (96d) — 상대적으로 균일
- SIFT 1.5M (128d) — 더 skewed

Selectivity

- 1%, 5%, 10%, 30%, 50% (RQ1/RQ2와 동일 5점)

반복

- 5-seed × 100 query (RQ1/RQ2와 동일)

측정 지표

- Q-error (hook_est vs true_card)
- paired Wilcoxon p-value

- Recovery Rate
-

구현 계획

Week 1 (4/28~5/4): Offline Partition 3종

- C (RandProj): 구현 ~2시간, 실험 ~2시간
- A (LSH): 구현 ~4시간, 실험 ~2시간
- E (Hilbert): 구현 ~4시간, 실험 ~2시간
- F (MiniBatch): 구현 ~1시간, 실험 ~2시간

Week 2 (5/5~5/11): Online + Weight

- G (DistShell): 구현 ~4시간, 실험 ~3시간
- B (KDE-pilot): 구현 ~6시간, 실험 ~4시간
- H (Importance): 구현 ~6시간, 실험 ~3시간

Week 3 (5/12~5/18): 분석

- Recovery Rate 비교 그래프
- 패러다임 간 cross-analysis
- 비용-효과 분석 (사전 학습 시간 vs recovery)

Week 4 (5/19~5/27): 최종 발표 준비

- 최종 보고서 작성
 - 발표 슬라이드
 - 포스터
-

논문 기여 3가지

1. 체계적 비교: 벡터 카디널리티 추정에서 distribution-agnostic 7가지 방법의 최초 체계적 비교
2. Recovery Rate 프레임워크: oracle(KM20) 대비 회수율 개념 도입, 비용-효과 정량 평가

3. Hilbert Curve 최초 적용: 공간 DB 기법(space-filling curve)을 벡터 카디널리티 추정에 적용

RQ1/RQ2와의 연결

RQ3의 모든 방법은 RQ1/RQ2에서 발견한 문제를 해결하는 연장선:

- **RQ1:** 밀도가 비균일하다 → uniform sampling이 나쁘다
- **RQ2:** 밀도 비균일을 파티션으로 교정 (KM20 총화)
- **RQ3 파티션 방법 (A/C/E/F):** KM20과 같은 교정을 사전 학습 없이 달성
- **RQ3 적응형 (B/G):** 쿼리 시점에 동적으로 교정
- **RQ3 가중치 (H):** 같은 문제를 다른 메커니즘(가중치)으로 교정 — 밀도가 높은 영역을 "덜 세고" 낮은 영역을 "더 세는" 효과

H(Importance Sampling)는 Control Variate 기법의 상위 개념이며, 총화와는 근본적으로 다른 분산 감소 경로. 둘 다 보여줘야 "밀도 비균일 교정"이라는 상위 프레이밍이 완성됨.

Exqutor Adaptive Sampling과의 관계

Exqutor 논문의 기존 Adaptive Sampling은 "이전 쿼리 결과로 sample_size를 모멘텀 기반 조정"하는 feedback 기반 학습. 이는 우리 RQ3의 Online Adaptive 패러다임에 해당하나:

1. Phase 7에서 Adaptive update path가 SIGSEGV 크래시 (design constraint #5)
2. 크래시 회피(update_sample_size=off)해도 hook_est 기반 STRAT가 BERN보다 나쁜 negative finding
3. Adaptive가 조정하는 건 sample_size(표본 크기)이지 sample_allocation(표본 배분)이 아님

따라서 우리 RQ3는 "Exqutor Adaptive의 한계를 인식하고, sample_allocation 차원의 대안을 제시"하는 positioning.

탈락 후보 상세

후보	탈락 이유
I. Density-Inverse Pre-weighting	H(Importance)의 offline 변형. 구현이 거의 동일하므로 독립 항목 불필요
J. Query-Adaptive Projection	G(Distance-Shell)의 열등 버전. 쿼리 방향 1D 사영 < 유클리드 거리 전체
K. Mini-batch PCA + Projection	C(RandProj)의 변형. 약간의 사전 학습으로 사영 방향 최적화. C 실험 시 비교 1줄로 처리
Latin Hypercube Sampling	고차원(96d+)에서 K^d 셀 폭발 → 차원 축소 필수 → C와 동치
Tree-based (VP/KD/Ball)	구축 비용이 k-means와 비슷. KM20과 차별 없음
Balanced Sampling (Cube)	이론적이나 구현 극난. 학부 캡스톤 범위 초과
Coreset Construction	샘플링이 아닌 부분집합 선택 문제. RQ3 범위 밖
Multi-armed Bandit	반복 쿼리 학습. 단일 쿼리 기준인 우리 세팅에 부적합
Optimal Transport	Wasserstein 기반. 이론적이나 구현 비현실적
Control Variate	H(Importance)의 특수 케이스. H에 통합

방법 분류 교차표 (정보 시점 × 포착 방식)

	직접 분할	좌표 변환	거리 기반	가중치 조정
Offline	F (MiniBatch)	C (RandProj), A (LSH), E (Hilbert)	—	—
Online	—	—	G (DistShell), B (KDE)	H (Importance)
None	RANDOM20	—	—	BERNOULLI

빈 셀(Offline×거리, Offline×가중치, Online×분할, Online×변환)은 기존 방법의 변형이거나 논리적으로 다른 셀에 귀결됨을 확인 완료.

맹점 점검 완료 사항

- 총화 프레임 간힘 → H(가중치)로 커버
- 실험 세팅 특수성 → B/G의 2-pass는 Python에서 해결
- 방법 간 조합 → 개별 먼저, 조합은 후속 연구
- 기대 효과 과대평가 → Hilbert/RandProj는 고차원 한계 가능, negative도 기여
- Feedback 기반 → Exqutor Adaptive가 커버, Phase 7 negative로 제외 근거
- 7가지 외 독립 방법 부재 확인 (I/J/K/LHS/Tree/Balanced/Coreset/MAB 전부 기존 7가지의 변형 또는 범위 밖)